



Web Back-End

By Jawad Jammoul

Content:

Back-End:

Chapter 01 :Node.js & Backend Development	1
Chapter 02 : php	13
Chapter 03 : Git & GitHub	46

Chapter 01: Node.js & Backend Development

01*Introduction to Backend Development:

What is Backend?

The backend is the part of a web application that runs on the server, not in the user's browser. It handles logic, data processing, authentication, and databases.

Simple Example: When a user logs in:

When a user logs in:

Frontend: sends email & password.

Backend: checks the database.

Backend: sends result (success / failure).

The user never sees the backend code.

Server-side Example:

```
if (password === "1234") {  
  res.end("Login successful");  
} else {  
  res.end("Login failed");  
}
```

Frontend vs Backend (Simple Flow Example):

Frontend (React):

```
fetch("http://localhost:3000/users")  
  .then(res => res.json())  
  .then(data => console.log(data));
```

Backend (Node.js):

- receives request
- fetches users from database
- sends users as JSON

Database (SQL):

```
SELECT * FROM users;
```

Role of Node.js:

- Node.js allows JavaScript to run outside the browser, so we can build servers and APIs.

Without Node.js:

- JavaScript works only in the browser.
- No server.
- No database access.

With Node.js:

- JavaScript runs on the server.
- Can create APIs.
- Can connect to databases.

Node.js Example (Simple Server):

```
const http = require("http");  
http.createServer((req, res) => {  
  res.end("Hello from backend");  
}).listen(3000);
```

Open browser:

```
http://localhost:3000
```

Without Database:	With Database:
<pre>let users = []; users.push("Jawad");</pre>	<pre>INSERT INTO users (name) VALUES ("Jawad");</pre>
Data disappears when the server stops.	Data is saved permanently. Can be used anytime.

02*What is Node.js:

JavaScript Runtime Environment:

Node.js is a JavaScript runtime environment built on Chrome's V8 engine.

It allows JavaScript to:

- Access files
- Create servers
- Communicate with databases

JavaScript in Browser Example:

```
alert("Hello");
```

JavaScript in Node.js Example:

```
console.log("Hello from Node.js");
```

Browser Example:

```
document.getElementById("title").innerText = "Hello";
```

Node.js Example:

```
const fs = require("fs");  
fs.writeFileSync("test.txt", "Hello File");
```

Server-side JavaScript:

Node.js runs JavaScript on the server, not on the user's device.

03*Installing Node.js & npm:

Installing Node.js:

Download from: <https://nodejs.org>

Install LTS version:

Check installation (Bash):

```
node -v  
npm -v
```

What is npm?

npm (Node Package Manager) manages libraries and tools.

Example:

```
npm install express
```

This installs Express.js inside your project.

npm Example (package.json)

```
{
  "name": "my-app",
  "dependencies": {
    "express": "^4.18.0"
  }
}
```

Create file: app.js

```
console.log("Node.js is running");
```

Run in terminal (Bash):

```
node app.js
```

Output

```
Node.js is running
```

04*Node.js Modules:

What is a Module in Node.js?

A module is a reusable piece of code.

Node.js uses modules to organize code into small, manageable files.

Node.js has:

Built-in modules

Custom modules

Third-party modules (via npm)

Built-in Modules

Built-in modules come with Node.js, no installation needed.

Examples:

fs: File system

path: File paths

http: Create servers

require() is used to import a module.

```
const fs = require("fs");
```

This gives access to file system functions.

module.exports is used to export code from one file to another.

Custom Module Example (math.js):

math.js:

```
function add(a, b) {
  return a + b;
}
module.exports = add;
```

app.js:

```
const add = require("./math");
console.log(add(2, 3));
```

Output

```
5
```

Common Modules:

fs (File System)

Used to read and write files.

```
const fs = require("fs");
fs.writeFileSync("data.txt", "Hello Node.js");
```

path: Used to handle file paths safely.

```
const path = require("path");
console.log(path.basename(__filename));
```

http: Used to create servers.

```
const http = require("http");
```

05*Creating a Server with Node.js:

What is a Server?

A server is a program that:

- Listens for requests
- Sends responses

Browser → Request → Server → Response

http.createServer() : Creates an HTTP server.

Basic Server Example:

```
const http = require("http");
const server = http.createServer((req, res) => {
  res.end("Hello from Node.js server");
});
server.listen(3000);
```

Open browser: <http://localhost:3000>

Request & Response:

req: information sent from client

res: data sent back to client

Request Example:

```
console.log(req.url);
```

Response Example:

```
res.end("Response sent");
```

Running a Server on a Port:

A port is a communication endpoint.

Common ports:

3000 (development)

80 (HTTP)

443 (HTTPS)

Server Running Message:

```
server.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

06*Express.js Basics:

Why Express.js?

Using http directly:

Too much code

Hard routing

Manual handling

Express.js simplifies everything.

Installing Express: (Bash)

```
npm install express
```

Creating an Express Server:

```
const express = require("express");
const app = express();
app.listen(3000, () => {
  console.log("Express server running");
});
```

Routing (GET, POST) : Routes define how the server responds to requests.

GET Route Example:

```
app.get("/", (req, res) => {
  res.send("Home Page");
});
```

POST Route Example:

```
app.post("/login", (req, res) => {
  res.send("Login successful");
});
```

Full Express Example:

```
const express = require("express");
const app = express();
app.get("/", (req, res) => {
  res.send("Hello Express");
});
app.get("/users", (req, res) => {
  res.json(["Ali", "Sara", "Jawad"]);
});
app.listen(3000);
```

07*APIs&JSON:

What is an API?

An API (Application Programming Interface) is a way for programs to communicate with each other.

In web development:

Frontend (React) sends a request

Backend (Node.js) sends a response

Data is exchanged using JSON

Real-life Example:

A restaurant menu:

You order food (request)

Kitchen prepares it

You receive food (response)

API Example (Express):

```
app.get("/api/message", (req, res) => {  
  res.send("Hello from API");  
});
```

Browser: <http://localhost:3000/api/message>

REST API Basics:

REST API Basics: REST is a set of rules for designing APIs.

Common HTTP methods:

GET → get data

POST → send data

PUT → update data

DELETE → delete data

REST Example:

```
app.get("/users", (req, res) => {  
  res.json(["Ali", "Sara", "Jawad"]);  
});
```

What is JSON?

JSON (JavaScript Object Notation) is a lightweight data format.

Example:

```
{  
  "name": "Ali",  
  "age": 25,  
  "isStudent": true  
}
```

Sending JSON:

```
app.get("/user", (req, res) => {  
  res.json({  
    name: "Ali",  
    age: 25  
  });  
});
```

Receiving JSON:

```
app.use(express.json());  
app.post("/login", (req, res) => {  
  console.log(req.body);  
  res.send("Data received");  
});
```

08*Introduction to Databases:

What is a Database?

A database is a place to store data permanently.

Examples of stored data:

Users

Passwords

Products

Orders

Without databases: data disappears when server stops.

Why Databases Are Needed:

Save data
Search data
Update data
Handle large amounts of information
SQL vs NoSQL
SQL Databases
Structured data
Tables with rows and columns
Fixed schema

Examples:

MySQL
SQLite
NoSQL Databases
Flexible structure
JSON-like documents
MongoDB
Firebase

Why SQL Is Widely Used:

Strong structure:
- Reliable relationships
- Used in companies and banks
- Powerful queries

09*SQL Fundamentals:

Primary Key:

A primary key uniquely identifies each row.

Example:

id INT PRIMARY KEY

Rules:

Unique
Not null
One per table

Primary Key Example: (SQL)

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  age INT  
);
```

Relationships:

Tables can be connected together.

Example:

One user : many orders

Relationship Example: (SQL)

```
CREATE TABLE orders (  
  id INT PRIMARY KEY,  
  user_id INT,  
  product VARCHAR(50)  
);
```

user_id links orders to users

10*Basic SQL Commands:

These are the 4 most important SQL commands.

You will use them all the time.

SELECT : Used to read data from a table.

Example:

```
SELECT * FROM users;
```

Get all columns and all rows from users.

Select specific columns:

```
SELECT name, age FROM users;
```

Select with condition:

```
SELECT * FROM users WHERE age > 20;
```

INSERT : Used to add new data to a table.

Example:

```
INSERT INTO users (id, name, age)  
VALUES (1, 'Ali', 25);
```

Add a new user to the table.

UPDATE : Used to modify existing data.

Example:

```
UPDATE users  
SET age = 26  
WHERE id = 1;
```

Update Ali's age.

Without WHERE, all rows will be updated.

DELETE : Used to remove data.

Example:

```
DELETE FROM users WHERE id = 1;
```

Delete the user with id = 1.

11*Connecting Node.js with SQL:

Database Drivers:

Node.js cannot talk to SQL directly.
It needs a driver (a library).

Common drivers:

MySQL : mysql2
PostgreSQL : pg
SQLite : sqlite3

Installing MySQL Driver (Bash):

```
npm install mysql2
```

Creating a Database Connection:

Example (MySQL): (JS)

```
const mysql = require("mysql2");
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "test_db"
});
db.connect((err) => {
  if (err) {
    console.log("Connection failed");
  } else {
    console.log("Connected to database");
  }
});
```

What This Does:

Connects Node.js to the database
Keeps the connection open
Allows SQL queries

12*Using SQL Queries in Node.js:

Now Node.js can send SQL commands.

SELECT Example in Node.js: (JS)

```
db.query("SELECT * FROM users", (err, result) => {
  if (err) {
    console.log(err);
  } else {
    console.log(result);
  }
});
```

What Happens Here?

Node.js sends SQL query
Database executes it
Result comes back as JavaScript object

INSERT Example: (JS)

```
db.query(
  "INSERT INTO users (name, age) VALUES ('Sara', 22)",
  (err, result) => {
    console.log("User added");
  }
);
```

UPDATE Example: (JS)

```
db.query(
  "UPDATE users SET age = 23 WHERE name = 'Sara'",
  (err, result) => {
    console.log("User updated");
  }
);
```

DELETE Example: (JS)

```
db.query(
  "DELETE FROM users WHERE name = 'Sara'",
  (err, result) => {
    console.log("User deleted");
  }
);
```

Very Important Concept:

Node.js = logic

SQL = data

Driver = bridge between them

Node.js : sends response

React: displays result

13*Environment variables:

What are Environment Variables?

Environment variables are hidden values used by the application, such as:

Database password

Database username

API keys

Secret tokens

They should NOT be written directly in code.

Why They Are Important:

Bad:	Good:
password: "123456"	password: process.env.DB PASSWORD

.env File:

A.env file stores environment variables.

Example .env:

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=secret1 23
DB_NAME=test_db
```

Installing dotenv: (Bash)

```
npm install dotenv
```

Using .env in Node.js:

```
require("dotenv").config();
const db = mysql.createConnection({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME
});
```

Important Rule:

Never upload .env to GitHub.

Add it to .gitignore.

14*Error Handling:

Why Error Handling Is Important?

Errors WILL happen:

```
Database disconnected
Wrong SQL query
Server crashes
Good error handling:
Prevents crashes
Helps debugging
Improves user experience
Handling Database Errors
```

JS Code:

```
db.query("SELECT * FROM users", (err, result) => {
  if (err) {
    console.error("Database error:", err);
    return;
  }
  console.log(result);
});
```

Handling Server Errors (Express):

```
app.get("/users", (req, res) => {
  try {
    res.json(users);
  } catch (error) {
    res.status(500).send("Server error");
  }
});
```

Sending Error Response to Frontend: (JS)

```
res.status(500).json({  
  message: "Something went wrong"  
});
```

Golden Rule:

Never send raw errors to users.
Log errors, send clean messages.

15*React + Node.js + SQL (Full Flow):

This is the MOST IMPORTANT PART.

Full System Overview:

React : UI

Node.js : logic

SQL : data storage

Step 1: React Sends Request (JS)

```
fetch("http://localhost:3000/users")  
  .then(res => res.json())  
  .then(data => console.log(data));
```

Step 2: Node.js Receives Request (JS)

```
app.get("/users", (req, res) => {  
  db.query("SELECT * FROM users", (err, result) => {  
    if (err) {  
      res.status(500).send("Database error");  
    } else {  
      res.json(result);  
    }  
  });  
});
```

Step 3: SQL Executes Query (SQL)

```
SELECT * FROM users;
```

Database returns data.

Step 4: Node.js Sends Data Back (JS)

```
res.json(result);
```

Step 5: React Displays Data (Jsx)

```
(users.map(user => (  
  <p key={user.id}>{user.name}</p>  
)))
```

Complete Mental Picture:

```
graph TD
    A[User clicks button] --> B[React sends request]
    B --> C[Node.js receives request]
    C --> D[SQL runs query]
    D --> E[Node.js returns JSON]
    E --> F[React updates UI]
```

This Is Why node.js Exists

React cannot talk directly to databases

Node.js protects the database

Node.js controls logic and security

Chapter 02 : php

01*Definition and Why Using php:

php (Hypertext Preprocessor) is a server-side scripting language used to create dynamic and interactive web pages. PHP code is executed on the web server, and the result is sent to the user's browser as HTML.

Why Use PHP?

- Open-source and free.
- Easy to learn and use.
- Can be embedded inside HTML.
- Supports databases such as MySQL.
- Used to build dynamic websites and web applications.
- Runs on most operating systems and web servers.

02*php Syntax:

PHP code starts with **<?php** and ends with **?>**

Every statement ends with a semicolon (;).

Example:

```
<?php
echo "Hello World!";
?>
```

php Inside an HTML File:

PHP can be written directly inside an HTML document.

```
<!DOCTYPE html>
<html>
<body>
<h2>My First PHP Page</h2>
<?php
echo "Welcome to PHP!";
?>
</body>
</html>
```

Variables Example:

Every variable starts with the \$ symbol.

Variable names are case-sensitive.

```
<?php
$color = "Blue";
$Color = "Red";
echo $color;
echo "<br>";
echo $Color;
?>
```

Output:

```
Blue
Red
```

Note: \$color and \$Color are two different variables.

echo and print:

Both are used to display output.

echo:

Can display one or more values.

Slightly faster.

Does not return a value.

```
<?php
echo "Hello";
echo "<br>";
echo "Welcome";
?>
```

print:

Displays only one value.

Returns 1, so it can be used in expressions.

```
<?php
print "Hello World!";
?>
```

03*PHP Variables & Data Types:

PHP Variables:

Variables are used to store data in a program. Every variable in PHP starts with the \$ symbol. PHP automatically determines the data type based on the assigned value, so no type declaration is required.

Example:

```
$name = "John";
$age = 22;
```

PHP Data Types:

String

Stores text or characters.

```
$message = "Hello";
```

Integer

Stores whole numbers without decimal points.

```
$age = 25;
```


Float (Double)

Stores numbers with decimal points.

```
$price = 19.99;
```

Boolean

Stores only two values: true or false.

```
$isStudent = true;
```

Array

Stores multiple values in a single variable.

```
$colors = array("Red", "Blue", "Green");
```

Object

Stores an instance of a class.

```
$student = new Student();
```

NULL

Represents a variable with no value.

```
$x = NULL;
```

04*Comments:

Comments make the code easier to understand and are ignored during execution.

Single-line Comment (//)

```
<?php
// This is a comment
echo "Hello";
?>
```

Single-line Comment (#)

```
<?php
# Another single-line comment
echo "PHP";
?>
```

Multi-line Comment (/* ... */)

```
<?php
/*
This is
a multi-line
comment.
*/
echo "Welcome";
?>
```

05*PHP Strings:

A string is a sequence of characters used to store text. A string can be written using double quotes (" ") or single quotes (' ').

Example:

```
$text = "Hello World";
echo $text;
```

Output:

```
Hello World
```

Common String Functions:

1.strlen() : Returns the length of a string.

```
echo strlen("Hello");
```

Output:

```
5
```

2.str_word_count() : Counts the number of words.

```
echo str_word_count("Hello World");
```

Output:

```
2
```

3.strrev() : Reverses a string.

```
echo strrev("Hello");
```

Output:

```
olleH
```

4.strpos() : Finds the position of a word or character.

```
echo strpos("Hello World", "World");
```

Output:

```
6
```

5.str_replace() : Replaces text with another text.

```
echo str_replace("World", "PHP", "Hello World");
```

Output:

```
Hello PHP
```

6.PHP Numbers & Casting:

PHP supports numeric data types such as Integer and Float.

Example:

```
$age = 20;  
$price = 15.75;  
echo $age;  
echo "<br>";  
echo $price;
```

Output:

```
20
```

```
15.75
```

Type Casting:

Casting converts a value from one data type to another.

Convert String to Integer

```
$x = "100";  
$number = (int)$x;  
echo $number;
```

Output:

```
100
```

Convert Integer to String:

```
$x = 50;  
$text = (string)$x;  
echo $text;
```

Output:

```
50
```

Convert Integer to Boolean:

```
$x = 1;  
$value = (bool)$x;  
var_dump($value);
```

Output:

```
bool(true)
```

07*php Math:

PHP provides many built-in mathematical functions.

abs() : Absolute Value

```
echo abs(-15);
```

Output:

```
15
```

sqrt() : Square Root

```
echo sqrt(25);
```

Output:

```
5
```

round() : Round a Number

```
echo round(4.7);
```

Output:

```
5
```

max() : Largest Value

```
echo max(10, 20, 30);
```

Output:

```
30
```

min() : Smallest Value

```
echo min(10, 20, 30);
```

Output:

```
10
```

pi() : Value of π

```
echo pi();
```

Output:

```
3.1415926535898
```

rand() : Random Number

```
echo rand(1, 10);
```

Output:

```
Random number between 1 and 10
```

08*PHP Constants & Magic Constants:

PHP Constants:

A constant is a fixed value that cannot be changed after it is defined.

Constants are created using define() or const.

Example using define():

```
define("SITE_NAME", "My Website");  
echo SITE_NAME;
```

Output:

```
My Website
```

Example using const:

```
const PI = 3.14159;  
echo PI;
```

Output:

```
3.14159
```

Note: Constants do not use the \$ symbol.

PHP Magic Constants:

Magic constants are predefined constants whose values depend on where they are used.

__LINE__ : Returns the current line number.

```
echo __LINE__;
```

Output:

```
(Current line number)
```

__FILE__ : Returns the full path and filename.

```
echo __FILE__;
```

Output:

```
C:\xampp\htdocs\index.php
```

__DIR__ : Returns the current directory.

```
echo __DIR__;
```

Output:

```
C:\xampp\htdocs
```

__FUNCTION__ : Returns the current function name.

```
function test() {  
    echo __FUNCTION__;  
}  
test();
```

Output:

```
test
```

__CLASS__ : Returns the current class name.

```
class Student {  
    public function show() {  
        echo __CLASS__;  
    }  
}  
$obj = new Student();  
$obj->show();
```

Output:

```
Student
```

__METHOD__ : Returns the current class method.

```
class Student {  
    public function show() {  
        echo __METHOD__;  
    }  
}  
$obj = new Student();  
$obj->show();
```

Output:

```
Student::show
```

09*PHP Operators:

Operators are symbols used to perform operations on variables and values.

1.Arithmetic Operators:

Used for mathematical calculations.

Operators: +, -, *, /, %

Example:

```
$a = 10;  
$b = 3;  
echo $a + $b;  
echo "<br>";  
echo $a % $b;
```

Output:

```
13  
1
```

2.Assignment Operators:

Used to assign or update variable values.

Operators: =, +=, -=, *=, /=, %=

Example:

```
$x = 10;  
$x += 5;  
echo $x;
```

Output:

```
15
```

3.Comparison Operators:

Used to compare two values. The result is either true or false.

Operators: ==, ===, !=, >, <, >=, <=

Example:

```
$x = 10;  
$y = 20;  
var_dump($x < $y);
```

Output:

```
bool(true)
```

4.Logical Operators:

Used to combine multiple conditions.

Operators: && (AND), || (OR), ! (NOT)

Example:

```
$age = 20;  
var_dump($age > 18 && $age < 30);
```

Output:

```
bool(true)
```

5.Increment and Decrement Operators:

Increase or decrease a variable by 1.

Increment (++):

```
$x = 5;  
$x++;  
echo $x;
```

Output:

6

Decrement (--):

```
$x = 5;  
$x--;  
echo $x;
```

Output:

4

10*PHP if / else / elseif :

Conditional statements allow PHP to execute different code based on whether a condition is true or false.

if Statement:

Executes a block of code only when the condition is true.

Syntax:

```
if (condition) {  
    // code  
}
```

Example:

```
$age = 20;  
if ($age >= 18) {  
    echo "Adult";  
}
```

Output:

Adult

if...else Statement:

Executes one block if the condition is true; otherwise, executes another block.

Syntax:

```
if (condition) {  
    // code  
} else {  
    // code  
}
```

Example:

```
$age = 15;  
if ($age >= 18) {  
    echo "Adult";  
} else {  
    echo "Minor";  
}
```

Output:

Minor

if...elseif...else Statement:

Checks multiple conditions. The first true condition is executed.

Syntax:

```
if (condition1) {  
    // code  
} elseif (condition2) {  
    // code  
} else {  
    // code  
}
```

Example:

```
$score = 85;  
if ($score >= 90) {  
    echo "Excellent";  
} elseif ($score >= 75) {  
    echo "Very Good";  
} else {  
    echo "Needs Improvement";  
}
```

Output:

Very Good

11*PHP switch:

The switch statement is used to execute one block of code from multiple choices. It is often used instead of multiple if...elseif statements when checking the same variable.

Syntax:

```
switch (expression) {  
    case value1:  
        // code  
        break;  
    case value2:  
        // code  
        break;  
    default:  
        // code  
}
```

Example:

```
$day = 3;  
switch ($day) {  
    case 1:  
        echo "Monday";  
        break;  
    case 2:  
        echo "Tuesday";  
        break;  
    case 3:  
        echo "Wednesday";  
        break;  
    default:  
        echo "Invalid Day";  
}
```

Output:

Wednesday

12*PHP Match:

The match expression is a modern alternative to switch (introduced in PHP 8). It compares values using strict comparison (===) and returns a value.

Syntax:

```
$result = match (expression) {  
    value1 => result1,  
    value2 => result2,  
    default => defaultResult,  
};
```

Example:

```
$day = 2;  
$result = match ($day) {  
    1 => "Monday",  
    2 => "Tuesday",  
    3 => "Wednesday",  
    default => "Invalid Day",  
};  
echo $result;
```

Output:

```
Tuesday
```

13*PHP Loops:

Loops are used to execute a block of code repeatedly as long as a condition is true.

while Loop:

The while loop repeats the code while the condition is true.

Syntax:

```
while (condition) {  
    // code  
}
```

Example:

```
$i = 1;  
while ($i <= 5) {  
    echo $i . " ";  
    $i++;  
}
```

Output:

```
1 2 3 4 5
```

do...while Loop:

The do...while loop executes the code at least once, then checks the condition.

Syntax:

```
do {  
    // code  
} while (condition);
```

Example:

```
$i = 1;  
do {  
    echo $i . " ";  
    $i++;  
} while ($i <= 5);
```

Output:

```
1 2 3 4 5
```


for Loop:

The for loop is used when the number of iterations is known.

Syntax:

```
for (initialization; condition; increment) {  
    // code  
}
```

Example:

```
for ($i = 1; $i <= 5; $i++) {  
    echo $i . " ";  
}
```

Output:

1 2 3 4 5

foreach Loop:

The foreach loop is used to iterate through arrays.

Example:

```
$colors = array("Red", "Blue", "Green");  
foreach ($colors as $color) {  
    echo $color . " ";  
}
```

Output:

Red Blue Green

14*PHP Functions:

A function is a reusable block of code that performs a specific task. Functions help organize code and avoid repetition.

Creating a Function:

Syntax:

```
function functionName() {  
    // code  
}
```

Example:

```
function greet() {  
    echo "Welcome to PHP!";  
}  
greet();
```

Output:

Welcome to PHP!

Function with Parameters:

Parameters allow values to be passed into a function.

Example:

```
function greet($name) {  
    echo "Hello " . $name;  
}  
greet("Ali");
```

Output:

Hello Ali

Example:

```
function add($a, $b) {  
    return $a + $b;  
}  
echo add(10, 5);
```

Output:

15

15*PHP Arrays:

An array is a special variable that can store multiple values in a single variable.

Indexed Array:

Stores values using numeric indexes starting from 0.

Example:

```
$colors = array("Red", "Blue", "Green");  
echo $colors[0];
```

Output:

Red

Associative Array:

Stores data as key => value pairs.

Example:

```
$student = array(  
    "name" => "Ali",  
    "age" => 20  
);  
echo $student["name"];
```

Output:

Ali

Multidimensional Array:

An array that contains one or more arrays.

Example:

```
$students = array(  
    array("Ali", 20),  
    array("Sara", 22)  
);  
echo $students[1][0];
```

Output:

Sara

Common Array Functions:

count() : Returns the number of elements in an array.

```
$colors = array("Red", "Blue", "Green");  
echo count($colors);
```

Output:

3

sort() : Sorts an array in ascending order.

```
$numbers = array(5, 2, 8, 1);  
sort($numbers);  
print_r($numbers);
```

Output:

Array ([0] => 1 [1] => 2 [2] => 5 [3] => 8)

array_push() : Adds one or more elements to the end of an array.

```
$colors = array("Red", "Blue");  
array_push($colors, "Green");  
print_r($colors);
```

Output:

```
Array ( [0] => Red [1] => Blue [2] => Green )
```

16*PHP Regular Expressions & Functions:

A Regular Expression (Regex) is a pattern used to search, match, or replace text in a string.

PHP provides several built-in functions for working with regular expressions.

preg_match() : Checks whether a pattern exists in a string.

Example:

```
$text = "Welcome to PHP";  
if (preg_match("/PHP/", $text)) {  
    echo "Match Found";  
}
```

Output:

```
Match Found
```

preg_match_all() : Finds all matches of a pattern in a string.

Example:

```
$text = "PHP is easy. PHP is powerful.";  
echo preg_match_all("/PHP/", $text);
```

Output:

```
2
```

preg_replace() : Replaces matched text with new text.

Example:

```
$text = "I like Java";  
echo  
preg_replace("/Java  
/", "PHP", $text);
```

17*PHP Forms:

Forms are used to collect information from users and send it to the server for processing.

Examples of form data:

- Name
- Email
- Password
- Comments
- Forms Handling

PHP handles form data using two main methods:

- GET Method
- Sends data through the URL.
- Data is visible in the browser address bar.
- Used for non-sensitive data.

HTML Example:

```
<form method="get" action="welcome.php">  
    Name:  
    <input type="text" name="name">  
    <input type="submit">  
</form>
```

PHP File:

```
<?php
echo $_GET["name"];
?>
```

Output:

Ali

POST Method:

Sends data inside the request body.

Data is not visible in the URL.

Used for sensitive information like passwords.

HTML Example:

```
<form method="post" action="welcome.php">
  Name:
  <input type="text" name="name">
  <input type="submit">
</form>
```

PHP File:

```
<?php
echo $_POST["name"];
?>
```

Output:

Ali

Form Validation:

Form validation checks that user input is correct before processing it.

Common validation steps:

Check if fields are empty.

Check if the email format is correct.

Remove unwanted characters.

Validate URLs.

Example:

```
$name = $_POST["name"];
if (empty($name)) {
    echo "Name is required";
} else {
    echo "Welcome " . $name;
}
```

Output:

Welcome Ali

Required Fields:

The required attribute prevents submitting an empty field.

Example:

```
<input type="text" name="name" required>
```

If the user leaves it empty:

Output:

Please fill out this field

Validate Email

PHP provides `filter_var()` to check email format.

Example:

```
$email = "test@gmail.com";  
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {  
    echo "Valid Email";  
} else {  
    echo "Invalid Email";  
}
```

Output:

```
Valid Email  
Validate URL
```

A URL can be checked using FILTER_VALIDATE_URL.

Example:

```
$url = "https://www.example.com";  
if (filter_var($url, FILTER_VALIDATE_URL)) {  
    echo "Valid URL";  
} else {  
    echo "Invalid URL";  
}
```

Output:

```
Valid URL
```

18*PHP Date and Time:

PHP provides built-in functions to work with dates and times.

date() Function:

The date() function formats a date or time.

Syntax:

```
date(format);
```

Example:

```
echo date("Y-m-d");
```

Output:

```
2026-07-15
```

Common formats:

Y : Four-digit year.

m : Month.

d : Day.

H : Hour.

i : Minutes.

s : Seconds.

Display Current Time:

```
echo date("H:i:s");
```

Output:

```
14:30:25
```

strtotime() Function:

Converts a date string into a timestamp.

Example:

```
$date = strtotime("tomorrow");  
echo date("Y-m-d", $date);
```

Output:

```
2026-07-16
```

mktime() Function:

Creates a timestamp for a specific date.

Example:

```
$date = mktime(0,0,0,7,15,2026);  
echo date("Y-m-d", $date);
```

Output:

```
2026-07-15
```

19*PHP Include:

The include statement allows inserting the content of one PHP file into another file.

It helps:

Reuse code.

Organize projects.

Avoid repeating the same code.

include

If the included file has an error, PHP shows a warning and continues execution.

Example:

```
header.php  
<h1>My Website</h1>  
index.php  
<?php  
include "header.php";  
echo "Home Page";  
?>
```

Output:

```
My Website  
Home Page  
require  
require works like include, but if the file is missing, the script stops.
```

Example:

```
<?php  
require "config.php";  
echo "Welcome";  
?>
```

20*PHP Files:

PHP can create, read, write, and manage files on the server.

Opening a File:

The **fopen()** function is used to open a file.

Syntax:

```
fopen(filename, mode);
```

Common modes:

r : Read only.

w : Write only (creates or replaces file).

a : Append data to the end.

Reading a File:

fread() reads content from a file.

Example:

```
$file = fopen("test.txt", "r");  
echo fread($file, filesize("test.txt"));  
fclose($file);
```

Output:

File content here

Writing to a File:

fwrite() writes data into a file.

Example:

```
$file = fopen("test.txt", "w");  
fwrite($file, "Hello PHP");  
fclose($file);
```

Output in file:

Hello PHP

Checking If a File Exists:

file_exists() checks whether a file exists.

Example:

```
if (file_exists("test.txt")) {  
    echo "File exists";  
} else {  
    echo "File not found";  
}
```

Output:

File exists

21*PHP Sessions:

A session is a way to store user information on the server so it can be used across multiple pages.

Sessions are commonly used for:

- Login systems.
- Shopping carts.
- User preferences.
- Starting a Session

Before using session variables, the session must be started using session_start().

Example:

```
<?php  
session_start();  
$_SESSION["username"] = "Ali";  
echo $_SESSION["username"];  
?>
```

Output:

Ali

Accessing Session Data:

Session variables can be accessed from different pages.

Example:

```
<?php
session_start();
echo $_SESSION["username"];
?>
```

Output:

Ali

Destroying a Session:

The session_destroy() function removes all session data.

Example:

```
<?php
session_start();
session_destroy();
echo "Session ended";
?>
```

Output:

Session ended

22*PHP Filters:

PHP filters are used to validate and sanitize user input. They help improve security by removing unwanted data.

Common uses:

- Validate emails.
- Validate URLs.
- Clean user input.

filter_var() : The filter_var() function applies a filter to a value.

Syntax:

```
filter_var(variable, filter);
Validate Email
<?php
$email = "user@gmail.com";
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Valid Email";
} else {
    echo "Invalid Email";
}
?>
```

Output:

Valid Email
Validate URL

```
<?php
$url = "https://www.google.com";
if (filter_var($url, FILTER_VALIDATE_URL)) {
    echo "Valid URL";
} else {
    echo "Invalid URL";
}
?>
```

Output:

Valid URL

Sanitize String : Removes special characters from a string.

```
<?php
$text = "<h1>Hello</h1>";
echo filter_var($text, FILTER_SANITIZE_STRING);
?>
```

Output:

Hello

23*PHP Callback Functions:

A callback function is a function that is passed as an argument to another function and can be called later.

Callbacks are commonly used with array functions.

Using a Callback Function

Example:

```
<?php
function message($name) {
    echo "Hello " . $name;
}
function process($callback) {
    $callback("Ali");
}
process("message");
?>
```

Output:

Hello Ali

Callback with Array Functions:

array_map() applies a function to every element of an array.

Example:

```
<?php
function double($x) {
    return $x * 2;
}
$numbers = array(1, 2, 3);
$result = array_map("double", $numbers);
print_r($result);
?>
```

Output:

Array ([0] => 2 [1] => 4 [2] => 6)

24*PHP JSON:

JSON (JavaScript Object Notation) is a format used to store and exchange data between applications. PHP provides functions to convert data between PHP arrays and JSON.

json_encode() : Converts a PHP value or array into JSON format.

Example:

```
<?php
$student = array(
    "name" => "Ali",
    "age" => 20
);
echo json_encode($student);
?>
```

Output:

```
{"name":"Ali","age":20}
```

json_decode() : Converts JSON data into a PHP object or array.

Example:

```
<?php
$json = '{"name":"Ali","age":20}';
$data = json_decode($json);
echo $data->name;
?>
```

Output:

```
Ali
```

Convert JSON to Array:

Using the second parameter true:

```
<?php
$json = '{"name":"Ali","age":20}';
$data = json_decode($json, true);
echo $data["age"];
?>
```

Output:

```
20
```

25*PHP Exceptions:

An exception is an event that occurs during program execution and interrupts the normal flow of the program.

Exceptions are used to handle errors in a controlled way instead of stopping the program suddenly.

try : Contains code that may cause an error.

throw : Creates an exception.

catch : Handles the exception.

Basic Exception Handling:

Example:

```
<?php
try {
    throw new Exception("Something went wrong");
}
catch (Exception $e) {
    echo $e->getMessage();
}
?>
```

Output:

```
Something went wrong
```

Using Multiple Exceptions:

Different errors can be handled using multiple catch blocks.

Example:

```
<?php
try {
    $number = 0;
    if ($number == 0) {
        throw new Exception("Cannot divide by zero");
    }
}
catch (Exception $e) {
    echo $e->getMessage();
}
?>
```

Output:

Cannot divide by zero

finally Block:

The finally block always executes whether an exception occurs or not.

Example:

```
<?php
try {
    echo "Try block";
}
catch (Exception $e) {
    echo "Error";
}
finally {
    echo " Finished";
}
?>
```

Output:

Try block Finished

26*PHP OOP (Object-Oriented Programming):

OOP is a programming style based on classes and objects. It helps organize code, reuse functionality, and make large applications easier to maintain.

Classes and Objects

Class : A class is a blueprint used to create objects. It defines properties (variables) and methods (functions).

Example:

```
<?php
class Student {
    public $name;
    function showName() {
        echo $this->name;
    }
}
?>
```

Object : An object is an instance of a class. It can access the properties and methods defined inside the class.

Example:

```
<?php
class Student {
    public $name;
    function showName() {
        echo $this->name;
    }
}
$student1 = new Student();
$student1->name = "Ali";
$student1->showName();
?>
```

Output:

Ali

Constructor:

Constructor : A constructor is a special method that runs automatically when an object is created. In PHP, the constructor is defined using `__construct()`.

Example:

```
<?php
class Student {
    public $name;
    function __construct($name) {
        $this->name = $name;
    }
    function show() {
        echo $this->name;
    }
}
$student = new Student("Ali");
$student->show();
?>
```

Output:

Ali

Destructor:

Destructor : A destructor is a special method that runs automatically when an object is destroyed or the script ends. It is defined using `__destruct()`

Example:

```
<?php
class Student {
    function __destruct() {
        echo "Object destroyed";
    }
}
$obj = new Student();
?>
```

Output:

Object destroyed

Access Modifiers:

Access modifiers control where class properties and methods can be accessed.

Public:

Can be accessed from anywhere.

Default access modifier.

Example:

```
public $name;
```

Private:

Can only be accessed inside the same class.

Example:

```
private $password;
```

Protected:

Can be accessed inside the class and its child classes.

Example:

```
protected $age;
```

PHP Class Constants:

A class constant is a value that cannot be changed after it is defined.

Constants inside classes are created using the const keyword.

Example:

```
<?php
class Student {
    const SCHOOL = "ABC University";
}
echo Student::SCHOOL;
?>
```

Output:

```
ABC University
```

27*PHP Inheritance and Polymorphism:

Inheritance:

Inheritance is an OOP concept that allows a class to inherit properties and methods from another class.

The existing class is called the parent class (base class).

The new class is called the child class (derived class).

In PHP, inheritance is created using the extends keyword.

Example:

```
<?php
class Person {
    public $name;
    function sayHello() {
        echo "Hello " . $this->name;
    }
}
class Student extends Person {
}
$student = new Student();
$student->name = "Ali";
$student->sayHello();
?>
```

Output:

```
Hello Ali
```

The Student class inherits the property \$name and the method sayHello() from the Person class.

Method Overriding:

A child class can redefine a method that already exists in the parent class.

Example:

```
<?php
class Person {
    function message() {
        echo "I am a person";
    }
}
class Student extends Person {
    function message() {
        echo "I am a student";
    }
}
$obj = new Student();
$obj->message();
?>
```

Output:

I am a student

Polymorphism:

Polymorphism means "many forms". It allows the same method name to perform different actions depending on the object that uses it.

In PHP, polymorphism is commonly achieved through:

- Method overriding.
- Interfaces.
- Abstract classes.

Example:

```
<?php
class Animal {
    function sound() {
        echo "Animal sound";
    }
}
class Dog extends Animal {
    function sound() {
        echo "Bark";
    }
}
class Cat extends Animal {
    function sound() {
        echo "Meow";
    }
}
$dog = new Dog();
$cat = new Cat();
$dog->sound();
echo "<br>";
$cat->sound();
?>
```

Output:

Bark
Meow

28*PHP Abstract Classes and Interfaces:

Abstract Classes:

An abstract class is a class that cannot be used to create objects directly. It is designed to be inherited by other classes.

An abstract class can contain:

Normal methods with code.

Abstract methods without code.

Abstract methods must be implemented by child classes.

Abstract Class Example:

```
<?php
abstract class Animal {
    abstract function sound();
    function sleep() {
        echo "Sleeping";
    }
}
class Dog extends Animal {
    function sound() {
        echo "Bark";
    }
}
$dog = new Dog();
$dog->sound();
echo "<br>";
$dog->sleep();
?>
```

Output:

```
Bark
Sleeping
```

Interfaces:

An interface defines a group of methods that a class must implement.

Interfaces contain method declarations only (without code).

A class uses an interface with the keyword implements.

A class can implement multiple interfaces.

Interface Example

```
<?php
interface Animal {
    public function sound();
}
class Dog implements Animal {
    public function sound() {
        echo "Bark";
    }
}
$dog = new Dog();
$dog->sound();
?>
```

Output:

```
Bark
```

29*PHP Traits:

A trait is a mechanism in PHP that allows code reuse between multiple classes.

Traits contain methods and properties that can be used by different classes.

A class uses a trait with the use keyword.

Traits solve the limitation that a class can inherit from only one parent class.

Example:

```
<?php
trait Message {
    function hello() {
        echo "Hello PHP";
    }
}
class Student {
    use Message;
}
$obj = new Student();
$obj->hello();
?>
```

Output:

Hello PHP

Using Multiple Traits:

A class can use more than one trait.

Example:

```
<?php
trait A {
    function showA() {
        echo "A";
    }
}
trait B {
    function showB() {
        echo "B";
    }
}
class Test {
    use A, B;
}
$obj = new Test();
$obj->showA();
$obj->showB();
?>
```

Output:

AB

Note: Traits are mainly used to share common code between unrelated classes.

30*PHP Static Methods and Static Properties:

Static Properties:

A static property belongs to the class itself, not to individual objects.

Created using the static keyword.

Accessed using ::.

Example:

```
<?php
class Counter {
    public static $count = 0;
}
echo Counter::$count;
?>
```

Output:

0

Static Methods:

A static method can be called without creating an object.

Created using the static keyword.

Accessed using ClassName::methodName().

Example:

```
<?php
class Calculator {
    public static function add($a, $b) {
        return $a + $b;
    }
}
echo Calculator::add(5, 3);
?>
```

Output:

8

Using Static Inside a Class:

Static methods cannot access normal properties using \$this.

self::\$property; is used to access static properties inside the same class.

31*PHP Namespaces:

A namespace is used to organize PHP code and prevent name conflicts between classes, functions, or constants with the same name.

Namespaces are useful in large projects where many files exist.

Creating a Namespace:

```
<?php
namespace School;
class Student {
    function show() {
        echo "Student Class";
    }
}
?>
```

Using a Namespace:

The use keyword imports a class from another namespace.

Example:

File 1:

```
namespace School;  
class Student {  
}
```

File 2:

```
use School\Student;  
$obj = new Student();
```

Accessing with Full Namespace:

```
$obj = new School\Student();
```

Note: Namespaces help avoid conflicts when two classes have the same name.

32*PHP Iterables:

An iterable is a data type that can be looped through using foreach.

In PHP, iterable includes:

- Arrays.
- Objects that implement the Traversable interface.

Using Iterable with Array:

Example:

```
<?php  
function printItems(iterable $items) {  
    foreach ($items as $item) {  
        echo $item . " ";  
    }  
}  
$colors = array("Red", "Blue", "Green");  
printItems($colors);  
?>
```

Output:

```
Red Blue Green
```

Iterable with Function Return Type:

A function can specify that it returns an iterable value.

Example:

```
<?php  
function getNumbers(): iterable {  
    return [1, 2, 3];  
}  
foreach(getNumbers() as $number) {  
    echo $number . " ";  
}  
?>
```

Output:

```
1 2 3
```

33*PHP MySQL Database:

PHP can be connected with MySQL databases to store, retrieve, update, and delete data dynamically. A database allows websites to save information permanently, such as:

- User accounts.
- Products.
- Orders.
- Posts.
- Messages.

PHP commonly uses MySQL with SQL (Structured Query Language) commands to communicate with the database.

Connecting PHP to MySQL:

PHP can connect to MySQL using:

- MySQLi extension
- PDO (PHP Data Objects)

Both allow PHP applications to execute SQL queries.

MySQLi Connection:

The mysqli class is used to create a connection between PHP and MySQL.

Syntax:

```
$conn = new mysqli(server, username, password, database);
```

Example:

```
<?php
$server = "localhost";
$username = "root";
$password = "";
$database = "test";
$conn = new mysqli($server, $username, $password, $database);
if ($conn->connect_error) {
    die("Connection failed");
}
echo "Connected successfully";
?>
```

Output:

```
Connected successfully
```

Creating a Database Connection Using PDO:

PDO provides a more flexible way to connect with different database systems.

Example:

```
<?php
try {
    $conn = new PDO(
        "mysql:host=localhost;dbname=test",
        "root",
        ""
    );
    echo "Connected successfully";
}
catch(PDOException $e) {
    echo "Connection failed";
}
?>
```

Output:

```
Connected successfully
```

PHP MySQL CRUD Operations:

CRUD represents the four main database operations:

Create : Insert new data.

Read : Retrieve data.

Update : Modify existing data.

Delete : Remove data.

1.INSERT Data (Create):

Used to add new records into a table.

```
INSERT INTO students(name, age)
VALUES('Ali',20);
PHP Example:
$sql = "INSERT INTO students(name, age)
VALUES('Ali',20)";
$conn->query($sql);
```

Database Result:

ID	Name	Age
1	Ali	20

2.SELECT Data (Read):

Used to reSQL:

```
SELECT * FROM students;
```

PHP Example:

```
$sql = "SELECT * FROM students";
$result = $conn->query($sql);
while($row = $result->fetch_assoc()) {
    echo $row["name"];
}
```

Output:

```
Ali
```

3.UPDATE Data:

Used to modify existing records.

```
UPDATE students
SET age=21
WHERE name='Ali';
PHP Example:
$sql =
"UPDATE students
SET age=21
WHERE name='Ali'";
$conn->query($sql);
Database Result:
Before:
Ali 20
After:
Ali 21
```

4.DELETE Data:

Used to remove records.

```
DELETE FROM students
WHERE name='Ali';
PHP Example:
$sql =
"DELETE FROM students
WHERE name='Ali'";
$conn->query($sql);
```

Output:

```
Record deleted
```

Prepared statements are used to protect databases from SQL Injection attacks.

Instead of directly inserting user input into SQL queries, values are sent separately.

Example:

```
$stmt = $conn->prepare(
"INSERT INTO students(name, age)
VALUES (?, ?)"
);
$stmt->bind_param("si", $name, $age);
$name = "Ali";
$age = 20;
$stmt->execute();
```

SQL Injection:

SQL Injection is a security problem where attackers insert harmful SQL commands through user input.

Example of unsafe code:

```
$sql =
"SELECT * FROM users WHERE name='$name'";
```

A user could enter SQL commands instead of a normal name.

Solution: Use prepared statements.

Fetching Data Methods:

When retrieving data:

fetch_assoc() :Returns data as an associative array.

Example:

```
$row["name"];
```

Output:

```
Ali
```

fetch_array() : Returns both numeric and associative indexes.

Example:

```
$row[0];
$row["name"];
```

num_rows : Returns the number of rows.

Example:

```
echo $result->num_rows;
```

Output:

```
5
```

34.PHP XML:

XML (eXtensible Markup Language) is a markup language used to store and exchange data between different systems.

Unlike HTML, XML does not focus on displaying data; it focuses on storing and transporting data.

XML is commonly used for:

Data exchange between applications.

Web services and APIs.

Configuration files.

Storing structured information.

XML Structure:

XML uses custom tags to describe data.

Example:

```
<?xml version="1.0"?>
<student>
  <name>Ali</name>
  <age>20</age>
  <major>Computer Science</major>
</student>
```

Reading XML Files in PHP:

PHP provides the SimpleXML extension to read XML documents easily.

The `simplexml_load_file()` function loads an XML file into an object.

Example:

XML file (student.xml):

```
<student>
  <name>Ali</name>
  <age>20</age>
</student>
```

PHP:

```
<?php
$xml = simplexml_load_file("student.xml");
echo $xml->name;
echo "<br>";
echo $xml->age;
?>
```

Output:

```
Ali
20
```

Accessing XML Elements:

XML elements can be accessed like object properties.

Example:

```
echo $xml->name;
```

Output:

```
Ali
```

For multiple elements:

XML:

```
<students>
<student>
  <name>Ali</name>
</student>
<student>
  <name>Sara</name>
</student>
</students>
```

PHP:

```
foreach($xml->student as $student){
  echo $student->name;
  echo "<br>";
}
```

Output:

```
Ali
Sara
```

Creating XML Using PHP:

PHP can create XML data using SimpleXMLElement.

Example:

```
<?php
$xml = new SimpleXMLElement("<student></student>");
$xml->addChild("name", "Ali");
$xml->addChild("age", "20");
echo $xml->asXML();
?>
```

Output:

```
<?xml version="1.0"?>
<student>
  <name>Ali</name>
  <age>20</age>
</student>
```

XML vs JSON:

XML uses custom tags. More verbose (larger size). Supports attributes. Common in older systems and some web services.

Example:

```
<name>Ali</name>
```

JSON uses key-value pairs. Smaller and faster. Commonly used in modern APIs.

Example:

```
{
  "name": "Ali"
}
```

Important XML Functions in PHP:

simplexml_load_file() : Loads an XML file.

```
$xml = simplexml_load_file("data.xml");
```

asXML() : Converts XML object into XML string.

```
echo $xml->asXML();
```

addChild() : Adds a new XML element.

```
$xml->addChild("name", "Ali");
```

Chapter 03 : Git & GitHub

01*What is Version Control?

Definition:

Version control is a system that records changes to files over time so that you can recall specific versions later.

It's like a time machine for your code.

Why it's important:

Track changes and history of a project

Revert to previous versions if something breaks

Collaborate with multiple developers without conflicts

Example (conceptual):

Imagine you have a file index.html:

Version 1: <h1>Hello World</h1>

Version 2: <h1>Hello Web Voyage</h1>

Version 3: <h1>Welcome to Web Development</h1>

A version control system lets you go back to any previous version whenever needed.

02*What is Git?

Definition:

Git is a popular distributed version control system. It allows multiple developers to work on the same project

simultaneously while keeping a full history of changes.

Key points:

-Local and distributed: each developer has a full copy of the project history

-Fast and reliable

-Works with branches to experiment safely without affecting main code

Example commands: (Bash)

```
git init # Initialize a Git repository in a folder
```

```
git status # Check the status of your files
```

```
git add index.html # Stage a file for committing
```

```
git commit -m "Update header" # Save changes with a message
```

Conceptual example:

you are writing code in a project folder. Git records each checkpoint you commit so you can always go back or share it with others.

03*What is GitHub?

Definition:

GitHub is a cloud-based platform that hosts Git repositories online. It allows developers to collaborate. Share and manage projects.

Why GitHub is useful:

- Backup your projects online
- Collaborate with other developers easily
- Track issues, pull requests, and contributions

Example workflow:

- Create a repository on GitHub
- Push your local Git commits to GitHub:

```
git remote add origin https://github.com/username/my-project.git
git push -u origin main
```

Collaborators can clone, make changes, and push updates

Conceptual example:

GitHub is like a library of your projects where you can safely store them, show your work, and work with a team anywhere in the world.

04*Installing Git:

Definition:

Git needs to be installed on your computer before you can use it. It works on Windows, macOS, and Linux.

How to install:

Windows: Download from git-scm.com and run the installer.

macOS: Install via Homebrew.

Linux: Install via package manager.

Verify installation:

```
git --version
```

This should show the installed Git version.

05*Basic Git Workflow:

Concept:

Git workflow is the sequence of steps you use to track and manage changes in your project

Typical workflow steps:

Initialize a repository (if not already):

```
git init
```

Check status of files:

```
git status
```

Stage files for commit:

```
git add filename.txt
git add. # add all files
```

Commit changes with a message:

```
git commit -m "Initial commit"
```

Push changes to remote repository (GitHub):

```
git push -u origin main
```

Example scenario:

You create index.html and style.css. You add them to Git, commit, and push to GitHub. Later, if you make changes, you repeat the workflow to save new versions.

06*Creating a Repository:

Definition:

A repository (repo) is a storage space for your project, either locally or on GitHub.

Creating a local repository:

```
mkdir my-project  
cd my-project  
git init
```

This creates a .git folder that tracks all changes in the project.

Creating a GitHub repository:

- Go to GitHub: New Repository
- Give it a name (e.g., web-voyage)
- Choose Public or Private
- Click Create Repository

Connecting local repo to GitHub:

```
git remote add origin  
git branch -M main  
git push -u origin main
```

Conceptual example:

Local repo: your personal working folder

GitHub repo: online backup & collaboration space

07*Tracking Changes:

Definition:

Tracking changes in Git means monitoring which files have been modified, added, or deleted since commit.

Example scenario:

You have index.html. You add a new paragraph:

```
<p>Welcome to Web Voyage</p>
```

```
git status  
# Shows: modified: index.html  
git diff  
# Shows the exact line added
```

Concept:

Tracking lets you review changes before saving them permanently with a commit.

08*Branches:

Definition:

Branches allow you to develop features or experiment in isolation without affecting the main code (usually main branch).

Why use branches:

- Safe testing
- Parallel development
- Organized workflow

Commands:

```
git branch feature-navbar # create a branch
git checkout feature-navbar # switch to branch
git branch # list branches
```

Example scenario:

main branch: stable website

feature-navbar branch: trying a new navigation bar

Changes in feature-navbar won't affect main until merged

09*Merging Branches:

Definition:

Merging means combining changes from one branch into another, usually from a feature branch into main.

Commands:

```
git checkout main # switch to main branch
git merge feature-navbar # merge changes from feature-navbar
```

Example scenario:

You finished the navigation bar in feature-navbar. After testing, you merge it into main. Now main has the new navbar.

Handling conflicts:

If two branches changed the same line, Git will ask you to resolve the conflict manually before completing the merge.

10*Remote Repositories:

Definition:

A remote repository is a version of your project hosted online, typically on platforms like GitHub, Git or Bitbucket. It allows you to collaborate with others and backup your code.

Common commands:

```
git remote -v # Show remote repositories
git remote add origin https://github.com/username/web-voyage.git
git push -u origin main
git pull origin main
```

Example scenario:

You worked locally on index.html

Pushed your changes to GitHub with git push

Another developer can pull your changes with git pull

11*GitHub Workflow:

Definition:

GitHub workflow is a standard process for collaborating on projects using Git and GitHub.

Steps:

Clone repository:

```
git clone https://github.com/username/web-voyage.git
```

Create a new branch:

```
git checkout -b feature-header
```

Make changes locally:

Commit changes:

```
git add.  
git commit -m "Add new header section"
```

Push branch to GitHub:

```
git push origin feature-header
```

Create Pull Request (PR) on GitHub for review

Merge PR into main branch after approval

Conceptual example:

Multiple developers can work simultaneously on different branches and merge safely without overwriting each other's work.

12*gitignore:

Definition:

A .gitignore file tells Git which files or folders to ignore and not track in version control.

Why it's important:

Avoid committing sensitive information (e.g., passwords, APNkeys)

Skip unnecessary files (e.g., node modules, build folders)

Example gitignore file:

```
node modules/  
.env  
dist/  
*log
```

Example scenario:

You have node_modules folder and .env file for environment variables

By adding them to .gitignore, Git won't track them, keeping the repo clean and secure

13*Common Git Commands:

git init	Initialize a local repository
git clone <url>	Copy a remote repository locally
git status	Check changes and status
git add <file>	Stage files for commit
git commit -m "message"	Save changes
git push	Send changes to remote
git pull	Fetch & merge from remote
git branch <name>	Create a branch
git checkout <branch>	Switch branch
git merge <branch>	Merge branch into current branch

14*Why Git is Essential for Developers:

- Tracks changes and allows reverting to previous versions
- Enables safe collaboration with multiple developers
- Provides remote backup
- Organizes features, fixes, and experiments using branches

Example:

Restore a deleted file:

```
git checkout HEAD~1 index.html
```